

AI CONTENT FACTORY

Stateful Multi-Agent Orchestration & Evaluation Engine
FINAL YEAR PROJECT (FYP) — COMPREHENSIVE VIVA DEFENSE GUIDE

Candidate Project: AI Content Factory

Date of Viva: September 2026

Focus Domain: Multi-Agent Systems, RAG & Databases

Primary Backend: FastAPI + LangGraph + SQLite/ChromaDB

1. Exhaustive Database Architecture (Primary Defense Focus)

In this project, the data persistence tier is deliberately decoupled into **three distinct, specialized storage layers**. Each layer serves a fundamentally different purpose, handles different concurrency characteristics, and uses dedicated storage algorithms:

Storage Layer	Technology / Engine	Primary Responsibility	File / Location
Relational Jobs Database	SQLite 3 (WAL mode) / PostgreSQL	CRUD metadata for jobs, topics, configs, final markdown, SEO scores, media file links, and status transitions.	Agents_backend/data/web_jobs.db
State Graph Checkpointer	LangGraph SqliteSaver / PostgresSaver	Binary state persistence, execution snapshots at every graph step, crash recovery, and Human-in-the-Loop (HITL) outline review resumption.	Agents_backend/data/checkpoints.db
Vector Embedding Database	ChromaDB (HNSW index) / Chroma Cloud	Dense vector indexing of document chunks (PDF/DOCX/TXT) using 1536-dim OpenAI embeddings for cosine similarity RAG retrieval.	Agents_backend/data/chroma_db/

1.1 Relational Job Store (`web\_jobs.db`)

Table Name: web_jobs | **Primary Key:** id (TEXT / UUIDv4)

Key Columns: topic, tone, sections, status, created_at, completed_at, blog_folder, qa_score, qa_verdict, geval_scores (JSON), deepeval_scores (JSON), blog_file, podcast_file, video_file, plan_json (JSON), config_json (JSON).

Concurrency & Performance Engineering:

- **WAL Mode (Write-Ahead Logging):** Enabled via PRAGMA journal_mode=WAL;. In traditional SQLite rollback journals, writers block all readers and readers block writers. In WAL mode, SQLite writes new transactions to a separate .wal file. Readers read committed snapshots concurrently without locking out background worker threads writing progress updates.

- **Busy Timeout:** `PRAGMA busy_timeout=30000`; instructs the SQLite engine to wait up to 30 seconds for locks to clear instead of immediately throwing *database is locked* exceptions.
- **Synchronous Setting:** `PRAGMA synchronous=NORMAL`; ensures safety while eliminating synchronous disk flushes on every write, improving transaction speed tenfold.
- **Thread Safety & Connection Lifecycle:** Managed using Python's `@contextmanager` in `db.py` (guaranteeing `conn.close()`) combined with an in-memory `_db_lock = threading.Lock()` for schema migrations.
- **Zero-Downtime Dynamic Auto-Migration:** `init_db()` queries `PRAGMA table_info(web_jobs)` at application boot. If new columns (such as `geval_scores` or `image_quality`) are missing, it executes atomic `ALTER TABLE ADD COLUMN` statements on the fly.
- **Dual Database Engine Support:** When `DATABASE_URL` is present (e.g., in Docker containerized deployments), `db.py` automatically redirects queries to **PostgreSQL** using `psycopg` connection pooling and translates parameter placeholders from `?` to `%s` via `_format_sql()`.

1.2 LangGraph State Checkpointer (`checkpoints.db`)

Why is a State Checkpointer critical in Multi-Agent AI?

Traditional LLM applications are stateless or rely on volatile RAM (like LangGraph's `MemorySaver`). If a generation takes 5 minutes and the server reboots, all intermediate work is lost. More critically, Human-in-the-Loop (HITL) requires halting the graph at the Orchestrator stage so the user can review and edit the outline in the frontend. While waiting for human input (up to 20 minutes), the thread must persist state durably.

Mechanics of LangGraph `SqliteSaver` / `PostgresSaver`:

- **Thread Identification:** Every generation job receives a unique `thread_id = job_id` passed in `thread_cfg = {"configurable": {"thread_id": job_id}}`.
- **Internal Checkpoint Tables:** Managed automatically by `memory.setup()`, creating checkpoints and writes tables that store serialized state dictionaries (JSON/msgpack/binary blobs).
- **Interrupt After Orchestrator:** The graph is compiled with `interrupt_after=["orchestrator"]`. When the Orchestrator finishes planning the outline, LangGraph writes the state snapshot and pauses. The API reads the plan, pushes it to the UI via WebSocket, and sets status to `awaiting_approval`.
- **Seamless Resumption:** When the user approves or refines the outline, the backend invokes `graph.stream(None, thread_cfg)`. Passing `None` signals LangGraph to fetch the latest state from `checkpoints.db` and resume directly into the parallel Worker fan-out without re-running Router or Research!
- **Crash Recovery:** If the API process crashes mid-generation, calling `resume` immediately recovers the exact step reached, saving real API tokens and avoiding repeated web research.

1.3 ChromaDB Vector Store & Advanced RAG (`vector\_store.py`)

Why a Vector Database? Relational databases excel at exact scalar queries (e.g. `id = '...`'), but cannot calculate conceptual semantic similarity. ChromaDB stores high-dimensional dense vector embeddings representing the mathematical meaning of text.

Technical Specifications:

- **Collection Name:** `ai_content_factory_documents`
- **Embedding Model:** OpenAI `text-embedding-3-small` producing dense **1536-dimensional** floating point vectors.
- **Indexing Algorithm:** **HNSW** (Hierarchical Navigable Small World) graph index configured with Cosine Distance via `metadata={"hnsw:space": "cosine"}`. HNSW achieves logarithmic query complexity $O(\log N)$ for Approximate

Nearest Neighbor (ANN) searches.

- **Chunking & Ingestion Strategy:** Uploaded PDFs/DOCXs are parsed via pypdf or python-docx. The system performs character chunking (6,000 characters $\approx$ 1,500 tokens with 400 char overlap) and semantic chunking using cosine similarity boundaries across adjacent sentence embeddings.
- **Three-Tier Fail-Safe Retrieval:** (1) Query ChromaDB collection filtered by upload_id; (2) In-memory NumPy cosine similarity calculation if Chroma server has transient issues; (3) Fallback to extracted document evidence pool. This guarantees the pipeline never crashes on retrieval.

2. Complete End-to-End System Architecture

The system follows an asynchronous, decoupled client-server architecture. The backend engine coordinates a **stateful, cyclic directed graph** powered by LangGraph, FastAPI, and WebSocket streaming.

1. **Topic Guard & Validation:** Deterministic regex screen rejects empty/gibberish input; gpt-5-mini verifies semantic safety before job creation to protect API budget.
2. **Router Agent:** Evaluates prompt complexity to choose between closed_book (no search), hybrid (doc RAG + search), or open_book (web search only).
3. **Document Ingest / RAG:** Executes semantic chunking and ChromaDB HNSW indexing. Retrieves topic-focused evidence slices.
4. **Research Agent:** Fires parallel queries to Tavily Search API, scrapes full web content via Jina Reader (BeautifulSoup fallback), filters duplicate content via Jaccard similarity.
5. **Orchestrator Agent (HITL):** Plans outline (H2 titles, bullet points, word counts) and partitions evidence across sections. Graph interrupts execution for human outline approval.
6. **Parallel Worker Fan-Out:** LangGraph Send() primitive spawns concurrent Worker agents. Each worker writes its section using ONLY its assigned evidence partition (preventing source-stuffing).
7. **Reducer Subgraph:** Deterministic Python module that joins sections in order, generates SEO metadata, and places AI-generated images (DALL-E 3 / Flux).
8. **Completion Validator:** Regex validation checking for missing headers and word count thresholds with automated repair routines.
9. **Quality Control Auditor:** Audits content for hallucinations. Runs a non-LLM citation verifier checking every URL against research evidence; flags critical issues.
10. **Revision Agent (Cyclic Loop):** Surgically rewrites only sections with critical errors. Loops back to QA Auditor (bounded at 2 revisions max to prevent infinite recursion).
11. **Keyword SEO Optimizer:** Deterministic density calculation; weaves underrepresented keywords into passages.
12. **G-Eval Scorecard:** In-house LLM judge grading Coherence (30%), Relevance (20%), Accuracy (30%), and Tone (20%) on a 1.0–5.0 scale.
13. **Parallel Media Fan-Out:** Forks simultaneously into Campaign Gen (LinkedIn/Twitter), Video Gen (MoviePy + Whisper captions + Pexels B-roll), and Podcast Studio (Gemini 2.5 Flash audio).
14. **On-Demand DeepEval Audit:** Independent academic audit (Liu et al. 2023) triggered via separate REST endpoint on demand.

3. Technology Stack Breakdown & Contribution

Examiners frequently ask: *'Why did you choose this specific library instead of alternative X?'* Here is the technical justification for every key tool:

Technology	Version	Role in Project	Why Chosen / Contribution
FastAPI	0.115+	Backend Web Framework	Asynchronous ASGI performance, automatic OpenAPI/Swagger docs, native WebSocket support for streaming live agent execution events to the UI.
LangGraph	1.2+	Multi-Agent Orchestrator	Provides stateful cyclic graph execution, checkpoint persistence, Send() parallel fan-out, and interrupt_after for Human-in-the-Loop workflows.
ChromaDB	1.5+	Vector Database	Embedded open-source vector store with native HNSW indexing, cosine similarity metrics, and local disk persistence without requiring a standalone daemon.
SQLite 3 (WAL)	Built-in	Relational & State Storage	Zero-config, serverless, embeddable. Configured with Write-Ahead Logging (WAL) and 30s busy timeout for high-concurrency background threads.
MoviePy 2 + FFmpeg	2.2+	Automated Video Rendering	Programmatic compositing of 9:16 vertical short videos, audio synchronization, stock B-roll clip trimming, and subtitle burning.
OpenAI Whisper	20250625	Audio Transcription & Alignment	Provides word-level timestamps on synthesized TTS audio to render synchronized, karaoke-style video captions.
DeepEval	4.2+	Academic LLM Evaluation	Implements the official G-Eval framework (Liu et al. 2023) using chain-of-thought grading for academic rigor and audit trails.
React 19 + Vite	19.0 / 6.2+	Frontend UI & Tooling	Ultra-fast HMR, component modularity, real-time WebSocket state management, interactive outline plan editor, and rich markdown rendering.
Tailwind CSS v4	4.1+	UI Styling Engine	Modern CSS theme variables, fluid responsive layouts, glassmorphic dark-mode aesthetics, and zero runtime CSS overhead.

4. AI Models & External APIs Catalog

The system harnesses specialized, state-of-the-art models matched to specific tasks to balance reasoning accuracy, execution latency, and cost:

Model / API	Provider	Pipeline Function	Key Characteristics
-------------	----------	-------------------	---------------------

gpt-5-mini / gpt-4o-mini	OpenAI	Routing, Research Extraction, Workers, QA, Revision, Campaigns	Fast reasoning, high context window, JSON mode structured outputs, highly cost-effective for parallel worker fan-out.
text-embedding-3-small	OpenAI	Semantic Chunking & Vector RAG	1536 embedding dimensions, high MTEB benchmark score, cosine similarity normalized.
gemini-2.5-flash	Google DeepMind	Gemini Podcast Studio	Native audio generation direct from model (voice 'Aode') — not synthetic robotic TTS, but human-like conversational inflection.
Gemini TTS & Whisper	Google & OpenAI	Video Voiceover & Word Alignment	Gemini synthesizes natural voiceover script; Whisper generates precise word-level SRT timestamps for subtitles.
DALL·E 3 / Flux	OpenAI / Pollinations	Blog Article Image Generation	3-provider fallback chain (OpenAI -> Gemini Imagen -> Pollinations Flux) ensures image generation never fails.
Tavily Search API	Tavily AI	Web Research Intelligence	Engineered specifically for AI agents: returns clean, structured search results and raw text without web scraping boilerplate.
Pexels Video API	Pexels	B-roll Video Retrieval	Queries royalty-free portrait (9:16) HD video clips matching the topic for background montage in video synthesis.

5. Top 15 Examiner Database & Architecture Questions (With Winning Answers)

These are the exact high-probability questions examiners ask during FYP vivas, paired with high-scoring, technically precise answers:

Q1: Why did you choose SQLite for this project instead of a full client-server database like MySQL or MongoDB?

Winning Answer: For our architecture, SQLite is an intentional engineering decision, not a compromise. First, SQLite is serverless and embeds directly into the Python process memory space, eliminating network socket latency (IPC) for state lookups. Second, with **WAL mode (Write-Ahead Logging)** enabled (`PRAGMA journal_mode=WAL`), readers do not block writers and writers do not block readers, providing excellent read/write concurrency for background worker threads. Third, our codebase is architected with dual-database capability in `db.py`: if `DATABASE_URL` is supplied, the entire persistence layer seamlessly redirects to **PostgreSQL** using `psycopg` connection pooling. Thus, SQLite powers self-contained local development, while PostgreSQL is ready for multi-container production.

Q2: What is WAL Mode in SQLite and why did you enable it?

Winning Answer: By default, SQLite uses a rollback journal where a write operation locks the entire database file, causing `sqlite3.OperationalError: database is locked` when multiple concurrent threads or background API workers execute. In WAL mode (Write-Ahead Logging), changes are written sequentially to a separate `.wal` file while the original database remains untouched. Readers can read committed database pages concurrently while the background worker appends writes. Combined with `PRAGMA busy_timeout=30000`, WAL mode completely eliminates locking crashes during parallel agent execution.

Q3: What is LangGraph's checkpointer database and why couldn't you just use Python global variables?

Winning Answer: Python global variables or in-memory state (like `MemorySaver`) are volatile: if the application server restarts, crashes, or scales across processes, all active state is wiped. Our system uses **SqliteSaver** (and **PostgresSaver**) pointing to `checkpoints.db`. It stores a binary snapshot of the entire graph state at each node transition keyed by `thread_id`. This enables two mission-critical capabilities: 1) **Human-in-the-Loop (HITL) Outline Control:** The graph pauses after the Orchestrator node (`interrupt_after=['orchestrator']`) and the thread yields. The user can take 15 minutes to review and edit the outline in the frontend. When submitted, the backend resumes from the checkpoint via `graph.stream(None, thread_cfg)`. 2) **Crash Recovery:** If the server crashes mid-pipeline, the job can be resumed from the exact last saved step without repeating expensive research or LLM calls.

Q4: Why do you need ChromaDB if you already have SQLite? Can't SQLite store vectors?

Winning Answer: SQLite is a relational B-Tree database optimized for exact match lookups ($O(\log N)$) and structured relational joins. It cannot calculate high-dimensional geometric distances. In our Advanced RAG pipeline, documents are embedded into **1536-dimensional vectors** using OpenAI's `text-embedding-3-small`. ChromaDB implements the **HNSW (Hierarchical Navigable Small World)** graph indexing algorithm. HNSW enables Approximate Nearest Neighbor (ANN) search across dense vector spaces in logarithmic time $O(\log N)$ using **Cosine Similarity**. Doing this in relational SQLite would require scanning every row ($O(N)$) and computing dot products in Python, which is computationally prohibitive on large corpora.

Q5: How does your RAG pipeline chunk documents and prevent hallucination?

Winning Answer: Ingestion follows a two-tier strategy in `document_ingest.py`: 1) Character chunking (6,000 characters with 400-character overlap) preserves paragraph context. 2) Semantic chunking calculates cosine similarity deltas across adjacent sentence embeddings to split text at natural topical shifts. Each chunk preserves metadata including `page_start` and `page_end`. To prevent hallucination, the Orchestrator partitions evidence across parallel workers so workers cite distinct sources. Furthermore, the **Quality Control Auditor** runs a deterministic Python citation verifier that parses all generated URLs and cross-references them against the original evidence; any ungrounded claim forces a revision.

Q6: Explain the database schema of `web\_jobs`. How did you handle unstructured data like configs and scores?

Winning Answer: The `web_jobs` table uses a hybrid structured-unstructured design: Standard scalar attributes (`id`, `topic`, `tone`, `sections`, `status`, `created_at`, `completed_at`, `blog_folder`) are strongly typed columns with indexes for fast sorting and filtering. Complex nested structures — such as `config_json` (generation settings), `plan_json` (outline tree), `geval_scores`, and `deepeval_scores` (academic evaluation rubrics) — are serialized as JSON strings. In `db.py`, helper functions automatically serialize and deserialize these fields, providing the schema flexibility of a document store like MongoDB with the relational integrity of SQLite/PostgreSQL.

Q7: How did you handle schema migrations when you added new features?

Winning Answer: Rather than requiring a heavy external migration framework like Alembic for our SQLite setup, db.py implements a lightweight, zero-downtime auto-migration pattern in `init_db()`. On server startup, it executes `PRAGMA table_info(web_jobs)`, inspects existing column names, and runs dynamic `ALTER TABLE web_jobs ADD COLUMN ...` statements for any new features (e.g. `geval_scores`, `deepeval_scores`, `config_json`). This ensures seamless backward compatibility with existing databases.

Q8: What happens if two users generate blogs at the exact same time? How does your system handle concurrency?

Winning Answer: Concurrency is handled across three layers: 1) **FastAPI & Asynchronous Event Loop:** FastAPI handles HTTP requests asynchronously on Uvicorn worker threads. 2) **Thread Isolation:** Each job runs inside a dedicated background worker thread with a unique `job_id` and `thread_id`. State is never shared between jobs. 3) **Database Level:** SQLite WAL mode allows concurrent readers while writes wait up to 30 seconds (`busy_timeout=30000`). In PostgreSQL deployments, connection pooling (`ConnectionPool(max_size=20)`) guarantees dedicated client connections for parallel transactions.

Q9: What is the LangGraph `Send()` API and how is it used in your parallel workers?

Winning Answer: `Send()` is LangGraph's dynamic map-reduce primitive for fan-out orchestration. After the outline is approved, the Orchestrator produces a list of section tasks. The graph edge uses `Send('worker', task_state)` to spawn `N` independent worker executions concurrently. Each worker receives only its assigned section task and its partitioned evidence slice. When all parallel workers complete, LangGraph collects their state outputs and forwards them to the **Reducer** node to assemble the complete draft.

Q10: Why is G-Eval run before media generators and not after?

Winning Answer: G-Eval evaluates the quality of the written article itself (Coherence, Relevance, Accuracy, Tone). Because Campaign, Video, and Podcast generation nodes depend on the validated, SEO-optimized text produced by the earlier nodes, running G-Eval immediately after the Keyword Optimizer ensures that media generators receive the finalized, scored content. It also allows media generators to execute in parallel fan-out without blocking evaluation.

Q11: What is the difference between In-house G-Eval and DeepEval in your project?

Winning Answer: Our project offers two complementary evaluation modes: 1) **In-House G-Eval (`geval\_scores`):** Runs automatically inside the graph pipeline using structured LLM outputs to grade sections on a 1.0–5.0 scale across 4 rubrics. The overall score is calculated via deterministic Python weights (30% Coherence, 20% Relevance, 30% Accuracy, 20% Tone). 2) **Official DeepEval (`deepeval\_scores`):** Runs **on demand** via a dedicated endpoint (`POST /api/jobs/{job_id}/run-deepeval`) using the Confident AI DeepEval library. It executes full Chain-of-Thought (CoT) evaluation (Liu et al. 2023) normalized to a 0.0–1.0 scale. It is kept on demand to avoid incurring 4 extra judge calls on every default generation.

Q12: How does the Gemini Podcast Studio work? Is it text-to-speech?

Winning Answer: It is not traditional robotic TTS (like gTTS or Windows Speech). It utilizes Google DeepMind's **Gemini 2.5 Flash native audio output** via the google-genai SDK. We prompt the model to generate spoken conversational audio directly (voice 'Aoede') in raw PCM/WAV format. The model produces natural breathing, conversational pauses, and speech inflections that conventional concatenative or parametric TTS engines cannot replicate.

Q13: How does the Video Synthesis pipeline work?

Winning Answer: Video generation in video.py is an automated multi-step pipeline: 1) The LLM writes a vertical short script with scene breakdowns and voiceover narration. 2) Voiceover audio is generated via Gemini TTS. 3) **OpenAI Whisper** processes the synthesized audio to generate exact word-level timestamps. 4) The **Pexels API** queries and downloads vertical (9:16) royalty-free B-roll footage matching the script topics. 5) **MoviePy** and **FFmpeg** composite the B-roll clips, synchronize background music, and render synchronized karaoke captions into a final MP4 video.

Q14: Explain the cyclic loop between QA Agent and Revision Agent. How do you prevent infinite loops?

Winning Answer: Unlike traditional acyclic DAGs, LangGraph allows cyclic graphs. The QA Agent audits the draft against the research evidence pool and identifies hallucinated claims or citation errors. If critical issues exist, the conditional edge routes to the **Revision Agent**, which surgically rewrites only the defective sections and routes back to the QA Agent. To prevent infinite loops and unbounded token consumption, the loop is strictly bounded by `MAX_REVISIONS = 2`. If the limit is reached, the pipeline accepts the best-effort draft and proceeds to the Keyword Optimizer.

Q15: What are the primary limitations of this system?

Winning Answer: As documented in our thesis, the key limitations are: 1) **Single-worker concurrency for HITL:** Approval events are held in an in-memory event dictionary; scaling across multiple server instances requires a distributed Redis queue. 2) **Prompt Injection defense:** Content scraped from the open web is treated as data, but advanced adversarial prompt injection defenses (e.g. instruction fencing) are identified as future scope. 3) **Authentication:** Access is secured via a shared-secret API key rather than full multi-tenant OAuth2 user accounts.